

Quantum Machine Learning

1 What is Quantum Machine Learning?

Classical ML finds patterns by tuning knobs on a model. QML does the same thing — except some of those knobs live inside a quantum circuit. That is genuinely it. Everything else is details.

2 Why is it Useful?

Two reasons people are excited:

Speed. Some quantum algorithms can search or factorise exponentially faster than any known classical algorithm. If a learning task reduces to one of those, you win.

Richer models. A quantum circuit on n qubits lives in a 2^n -dimensional space. A classical computer cannot even write down that space once n gets large enough. Maybe that extra room lets quantum models find patterns classical ones miss.

Neither of these is proven for ML specifically — yet. But the theoretical possibility is strong enough that Google, IBM, and most big quantum labs have active QML teams.

3 How Does a Quantum Model Work?

Almost all QML today uses one recipe called a **variational quantum circuit (VQC)**. Here is the intuition:

Analogy. A classical neural network takes data in, multiplies by weight matrices, applies activations, and spits out a prediction. A VQC takes data in, rotates qubits by tunable angles, measures, and spits out a number. Training both means adjusting the tunable parts until the output is useful.

The three steps of a VQC:

1. **Encode.** Turn your classical data (numbers, images, molecules) into qubit rotation angles. This is called a *feature map*.
2. **Process.** Run the data through a circuit $U(\theta)$ whose gates have free parameters θ . These are the “weights” you train.
3. **Measure.** Read out an expectation value. This is your model’s prediction.

Then you compute a loss, compute gradients, and update θ — exactly like gradient descent. The gradient on quantum hardware is computed using a neat trick called the **parameter-shift rule**: shift a gate angle by $\pm\pi/2$, run the circuit twice, subtract. No approximations needed.

	Classical Neural Net	VQC
Weights	Matrix entries	Gate rotation angles θ
Forward pass	Matrix multiplications	Quantum circuit + measurement
Gradient	Backpropagation	Parameter-shift rule
Training loop	Adam / SGD	Same optimisers work fine

4 Applications

Classification

Feed in a data point, encode it, measure, threshold the output. You get a classifier. People have trained VQCs on the Iris dataset, handwritten digits (small versions), and medical images. It works — though classical SVMs still give it a run for its money on small datasets.

Quantum Kernels

Here is a clever shortcut: instead of running the full model on quantum hardware, use the quantum circuit only to compute how similar two data points are (a kernel). Then hand that similarity matrix to a completely classical SVM.

Intuition. Imagine projecting your data into a space so high-dimensional that previously tangled classes become linearly separable. That is what a kernel does. A quantum circuit can project into a 2^n -dimensional space that no classical kernel can efficiently reach.

Chemistry and Drug Discovery

This is the most mature application and the most likely to see real-world impact soon. Simulating a molecule quantum-mechanically is exponentially hard for classical computers. But a quantum computer is already a quantum system — so it can simulate another quantum system naturally.

The algorithm is called **VQE (Variational Quantum Eigensolver)**. It finds the lowest energy state of a molecule. Drug companies use this to screen which molecules are worth synthesising in the lab.

Optimisation

Many real-world problems — delivery routes, portfolio allocation, chip design — are combinatorial optimisation problems. **QAOA (Quantum Approximate Optimisation Algorithm)** is a VQC designed specifically for these. It is not ML in the strict sense but sits right at the same intersection of circuits and data.

Generative Models

Quantum circuits naturally produce probability distributions over bitstrings (via the Born rule). You can train a circuit to match a target distribution and then sample from it. This is used for generating molecular structures and, more experimentally, for synthetic data generation.

Application	What the quantum part does	Maturity
Classification	Replaces forward pass	Proof of concept
Kernel methods	Computes similarity in big space	Theoretically solid
Chemistry (VQE)	Simulates molecule energy	Most mature
Optimisation (QAOA)	Searches solution space	Active experiments
Generative models	Samples from distribution	Early research

5 Limitations

There is **no proven quantum advantage in ML yet**. Every benchmark result so far has either been on tiny toy datasets or has been matched by a smarter classical algorithm shortly after. Quantum advantage for ML is a hypothesis, not a fact.

Current hardware is noisy. Quantum computers today have error rates around 10^{-3} per gate. Deep circuits accumulate errors. Most VQC experiments use fewer than 20 qubits and very shallow circuits.

Training can break. Deep quantum circuits suffer from *barren plateaus* — gradients shrink exponentially with circuit depth, and the training signal disappears. Active research area, no full solution yet.

None of this means QML is not worth learning. It means the field is young and the interesting problems are still open. That is actually the exciting part.

6 Resources

Resource	Type	Good for
pennylane.ai/qml	Interactive demos	Coding VQCs from scratch
PennyLane YouTube	Video lectures	Visual intuition on circuits
Cerezo et al. 2021 (arXiv)	Review paper	Rigorous but readable survey

Prerequisite chapters: Qubits & Superposition → Single-Qubit Gates → Bloch Sphere → Multi-Qubit Gates → this chapter.